

Semester	B.E. Semester VII – Computer Engineering
Subject	R programming
Subject In-charge	Prof. SDW

Student Name	Diksha Parulekar.
Roll Number	23102A0068.
Grade and Subject Teacher's Signature	

Experiment Number	08	
Experiment Title	High-Performance Big Data Analytics Using	
Resources / Apparatus Required	Hardware: Computer system	Software: Python
Description	This lab builds a scalable analytics workflow in R on the NYC TLC Yellow Taxi trip data, about [N] million trips from [Jan–Mar 2023], joined with the Taxi Zone Lookup Table. The data was cleaned by removing missing values, duplicates and invalid trips. It was then converted to proper types, with hour, day, weekday and month extracted. data.table was used for joins, filtering and aggregation to find demand patterns, top routes, revenue, fare–distance behaviour and payment trends. The same operations were implemented with apply/lapply/purrr::map, vectorised code and data.table, and benchmarked with microbenchmark. A month-wise heavy computation was run sequentially and in parallel with foreach and doParallel, and the results were visualised with ggplot2.	

Program & Output

•

•

The screenshot shows the RStudio environment with a script editor and a console. The script defines a function `make_synthetic` that generates synthetic data for a pickup dataset. The function uses random sampling to create a dataset with columns like `date`, `time`, `pickup_location`, `dropoff_location`, `fare`, `tip`, `tolls`, `payment_type`, and `fare_amount`. The script then applies this function to a list of months and checks for errors.

```
# Synthetic fallback with the same schema (only used if the TLC download is unreachable)
make_synthetic <- function(n, n_seed) {
  db <- as.Date(paste0(n, "-01")); nd <- as.integer(format(seq(db, by = "month", length.out = 2)[2] - 1, "%d"))
  hp <- c(2,1.5,1.8,0.8,1.5,3,5,4.5,4.5,5,5,5,6,5,7,5,8,7,6,5,4,3); hp <- hp/sum(hp)
  pick <- as.POSIXct(db, tz = "UTC") + (sample(0:(nd-1), n, TRUE)*86400 + sample(0:23, n, TRUE, prob = hp)*3600 +
    sample(0:3599, n, TRUE))
  dist <- round(rlnorm(n, 0.6, 0.3), 2); dur <- pmax(1, dist*4 + rexp(n, 1/4))
  fare <- round(pmax(3, 3 + 2.8*dist + rnorm(n, 0, 1.5)), 2)
  pay <- sample(c(1,2,3,4), n, TRUE, prob = c(.7,.25,.03,.02))
  tip <- ifelse(pay == 1, round(fare*runif(n, 0, .3), 2), 0)
  data.table(tpcp_pickup_datetime = pick, tpep_dropoff_datetime = pick + dur*60,
    passenger_count = sample(c(1,2,3,4,5,6, NA), n, TRUE, prob = c(.7,.35,.05,.03,.02,.02)),
    trip_distance = dist, PUlocationID = sample(1:263, n, TRUE, prob = runif(263)^3),
    DOlocationID = sample(1:263, n, TRUE, prob = runif(263)^3), payment_type = pay,
    fare_amount = fare, tip_amount = tip, tolls_amount = 0, total_amount = round(fare + tip + 0.5 + 2.5, 2))
}

parts_raw <- lapply(months, function(m) tryCatch({ message("Loading ", m); read_month[m] },
  error = function(e) { message(" download failed for ", m, ": ", conditionMessage(e)); NULL }))
using_synthetic <- any(sapply(parts_raw, is.null))
if (using_synthetic) { message("Using SYNTHETIC fallback data"); parts_raw <- lapply(months, make_synthetic) }
dt <- rbindlist(parts_raw); rm(parts_raw); invisible(gc())
cat("Loaded", format(nrow(dt), big.mark = ","), "rows | synthetic fallback:", using_synthetic, "\n")
```

The console output shows the execution of the script, including the loading of data for each month and the final summary statistics:

```
...
Loading 2023-01
  download failed for 2023-01: could not find function "all_of"
Loading 2023-02
  download failed for 2023-02: could not find function "all_of"
Loading 2023-03
  download failed for 2023-03: could not find function "all_of"
Using SYNTHETIC fallback data
Loaded 3,000,000 rows | synthetic fallback: TRUE
```

At the bottom of the interface, the text "4.2 Inspect structure, dimensions, data types and summary statistics" is visible, along with a button labeled "Variables" and a terminal icon.

```
2310A0068 -Diksha Parulekar_R programming_assignment 8.ipynb
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Dimensions: 3000000 11
Classes 'data.table' and 'data.frame': 3000000 obs. of 11 variables:
 $ tpep_pickup_datetime : POSIXct, format: "2023-01-17 09:20:58" "2023-01-05 23:17:27" ...
 $ tpep_dropoff_datetime: POSIXct, format: "2023-01-17 09:45:44" "2023-01-05 23:22:22" ...
 $ passenger_count      : num 1 1 1 1 1 1 1 1 1 ...
 $ trip_distance        : num 5.9 0.5 0.77 1.12 0.21 ...
 $ PULocationID         : int 133 123 75 112 29 62 158 251 176 142 ...
 $ DOLocationID         : int 177 225 25 164 147 149 150 131 84 186 ...
 $ payment_type         : num 2 2 1 1 2 1 1 1 2 ...
 $ fare_amount          : num 20.82 4.79 6.54 6.38 4.75 ...
 $ tip_amount           : num 0 0 1.81 0.39 0.23 ...
 $ tolls_amount         : num 0 0 0 0 0 0 0 0 0 ...
 $ total_amount         : num 23.82 7.79 11.35 9.77 7.98 ...
 - attr(*, "internal.selfref")=(<pointer: 0x5b59b2cdb3f0>)
 tpep_pickup_datetime tpep_dropoff_datetime passenger_count
Min. :2023-01-01 00:00:00 Min. :2023-01-01 00:05:24 Min. :1.000
1st Qu.:2023-01-24 09:52:11 1st Qu.:2023-01-24 10:07:04 1st Qu.:1.000
Median :2023-02-15 00:02:56 Median :2023-02-15 00:19:35 Median :1.000
Mean :2023-02-15 02:13:35 Mean :2023-02-15 02:28:30 Mean :1.571
3rd Qu.:2023-03-08 18:35:59 3rd Qu.:2023-03-08 18:51:07 3rd Qu.:2.000
Max. :2023-03-31 23:59:49 Max. :2023-04-01 02:26:22 Max. :6.000
NAs :60034

 trip_distance PULocationID DOLocationID payment_type
Min. : 0.020 Min. : 1 Min. : 1.0 Min. :1.00
1st Qu.: 0.990 1st Qu.: 68 1st Qu.: 69.0 1st Qu.:1.00
Median : 1.820 Median :132 Median :135.0 Median :1.00
Mean : 2.731 Mean :134 Mean :133.7 Mean :1.37
3rd Qu.: 3.340 3rd Qu.:200 3rd Qu.:197.0 3rd Qu.:2.00
Max. :151.510 Max. :263 Max. :263.0 Max. :4.00

 fare_amount tip_amount tolls_amount total_amount
Min. : 3.00 Min. : 0.00 Min. : 0 Min. : 6.00
1st Qu.: 5.78 1st Qu.: 0.00 1st Qu.: 0 1st Qu.: 9.32
Median : 8.31 Median : 0.64 Median : 0 Median : 12.16
Mean : 10.68 Mean : 1.12 Mean : 0 Mean : 14.80
3rd Qu.: 12.57 3rd Qu.: 1.58 3rd Qu.: 0 3rd Qu.: 16.91
Max. :430.14 Max. :79.99 Max. : 0 Max. :444.52

Missing values per column:
 tpep_pickup_datetime tpep_dropoff_datetime passenger_count trip_distance
1: <int> <int> <int> <int>
 PULocationID DOLocationID payment_type fare_amount tip_amount tolls_amount
1: <ints> <ints> <ints> <ints> <ints> <ints>
```

23102A0068_Diksha Parulekar_R_programming_assignment 8.ipynb

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

Memory (raw): 228.9 Mb

4.3 Missing, duplicate, inconsistent and invalid records

```
[ ] clean_log <- data.table(step = "Raw rows", rows = nrow(dt))
log_step <- function(label) clean_log <- rbind(clean_log, data.table(step = label, rows = nrow(dt)))

dt <- dt[!is.na(passenger_count) & !is.na(payment_type)]; log_step("Dropped rows with missing passenger_count / payment_type")
dt <- unique(dt); log_step("Removed exact duplicate rows")
dt[, duration_min := as.numeric(difftime(tpep_dropoff_datetime, tpep_pickup_datetime, units = "mins"))]
dt <- dt[ trip_distance > 0 & trip_distance <= 100 &
  fare_amount > 0 & fare_amount <= 500 &
  total_amount > 0 & total_amount <= 1000 & tip_amount >= 0 &
  duration_min >= 1 & duration_min <= 180 &
  passenger_count %between% c(1, 6) &
  PULocationID %between% c(1, 263) & DOLocationID %between% c(1, 263) &
  year(tpep_pickup_datetime) == YEAR & month(tpep_pickup_datetime) %in% seq_len(N_MONTHS)]
log_step("Removed invalid / inconsistent rows (distance, fare, total, duration, passengers, zone IDs, out-of-range dates)")
clean_log[, removed := shift(rows, 1) - rows]
print(clean_log)

...
1:
2:
3:
4: Removed invalid / inconsistent rows (distance, fare, total, duration, passengers, zone IDs, out-of-range dates)
   rows removed
   <int> <int>
1: 3000000 NA
2: 2939966 60034
3: 2939966 0
4: 2939998 568

step
<char>
Raw rows
Dropped rows with missing passenger_count / payment_type
Removed exact duplicate rows
```

+ Code + Text

4.4 Data types, temporal features and memory reduction

[] mem_before <- object.size(dt)

[] Variables Terminal

23102A0068_Diksha Parulekar_R programming_assignment 8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

4: 2939398 568

4.4 Data types, temporal features and memory reduction

```

1 mem_before <- object.size(dt)
2 dt[, `:=`(passenger_count = as.integer(passenger_count),
3           PULocationID = as.integer(PULocationID), DOLocationID = as.integer(DOLocationID),
4           payment_type = factor(payment_type, levels = 0:6,
5                                 labels = c("Flex fare", "Credit card", "Cash", "No charge", "Dispute", "Unknown", "Voided trip"))
6 dt[, `:=`(pickup_date = as.Date(tpep_pickup_datetime),
7           pickup_hour = hour(tpep_pickup_datetime),
8           pickup_day = mday(tpep_pickup_datetime),
9           dow = factor(wday(tpep_pickup_datetime), levels = 1:7, labels = c("Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat")),
10          pickup_month = month(tpep_pickup_datetime))]
11 dt[, month_name := factor(pickup_month, levels = 1:12, labels = month.abb)]
12 # drop what is no longer needed
13 dt[, c("tpep_pickup_datetime", "tpep_dropoff_datetime", "tolls_amount") := NULL]
14 invisible(gc())
15 cat("Memory before:", format(mem_before, units = "MB"), "-> after:", format(object.size(dt), units = "MB"), "\n")
16 str(dt)

```

```

... Memory before: 246.7 Mb -> after: 224.3 Mb
Classes 'data.table' and 'data.frame': 2939398 obs. of 15 variables:
 $ passenger_count: int 1 1 1 1 1 1 1 1 1 1 ...
 $ trip_distance : num 5.9 0.5 0.77 1.12 0.21 ...
 $ PULocationID : int 133 123 75 112 29 62 158 251 176 142 ...
 $ DOLocationID : int 177 225 25 164 147 149 150 131 84 186 ...
 $ payment_type : Factor w/ 7 levels "Flex fare","Credit card",...: 3 3 2 2 2 3 2 2 3 ...
 $ fare_amount : num 20.82 4.79 6.54 6.38 4.75 ...
 $ tip_amount : num 0 0 1.81 0.39 0.23 ...
 $ total_amount : num 23.82 7.79 11.35 9.77 7.98 ...
 $ duration_min : num 24.77 4.92 11.3 8.44 13.62 ...
 $ pickup_date : IDate, format: "2023-01-17" "2023-01-05" ...
 $ pickup_hour : int 9 23 11 11 20 15 13 18 0 12 ...
 $ pickup_day : int 17 5 1 25 10 4 18 26 17 15 ...
 $ dow : Factor w/ 7 levels "Sun","Mon","Tue",...: 3 5 1 4 3 4 4 5 3 1 ...
 $ pickup_month : int 1 1 1 1 1 1 1 1 1 1 ...
 $ month_name : Factor w/ 12 levels "Jan","Feb","Mar",...: 1 1 1 1 1 1 1 1 1 1 ...
 - attr(*, "internal.selfref")=pointer: 0x5b59b2c3db3f0

```

Variables Terminal

30°

Search

23102A0068_Diksha Parulekar_R programming_assignment 8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

dt[zones, `:=`(PU_Borough = 1.Borough, PU_Zone = 1.Zone), on = .(PULocationID = LocationID)]

dt[zones, `:=`(DO_Borough = 1.Borough, DO_Zone = 1.Zone), on = .(DOLocationID = LocationID)]

print(head(dt, 5))

cat("Rows with unmatched pickup zone:", dt[is.na(PU_Zone), .N], "\n")

```

... Zones: 265
passenger_count trip_distance PULocationID DOLocationID payment_type
<int> <num> <int> <int> <fctr>
1: 1 5.90 133 177 Cash
2: 1 0.50 123 225 Cash
3: 1 0.77 75 25 Credit card
4: 1 1.12 112 164 Credit card
5: 1 0.21 29 147 Credit card
fare_amount tip_amount total_amount duration_min pickup_date pickup_hour
<num> <num> <num> <num> <IDate> <int>
1: 20.82 0.00 23.82 24.773334 2023-01-17 9
2: 4.79 0.00 7.79 4.924426 2023-01-05 23
3: 6.54 1.81 11.35 11.298798 2023-01-01 11
4: 6.38 0.39 9.77 8.443891 2023-01-25 11
5: 4.75 0.23 7.98 13.619556 2023-01-10 20
pickup_day dow pickup_month month_name PU_Borough PU_Zone
<int> <fctr> <int> <fctr> <char> <char>
1: 17 Tue 1 Jan Brooklyn Kensington
2: 5 Thu 1 Jan Brooklyn Homecrest
3: 1 Sun 1 Jan Manhattan East Harlem South
4: 25 Wed 1 Jan Brooklyn Greengpoint
5: 10 Tue 1 Jan Brooklyn Brighton Beach
DO_Borough DO_Zone
<char> <char>
1: Brooklyn Ocean Hill
2: Brooklyn Stuyvesant Heights
3: Brooklyn Boerum Hill
4: Manhattan Midtown South
5: Bronx Longwood
Rows with unmatched pickup zone: 0

```

5. Task 2: Transportation Data Analytics (data.table)

```

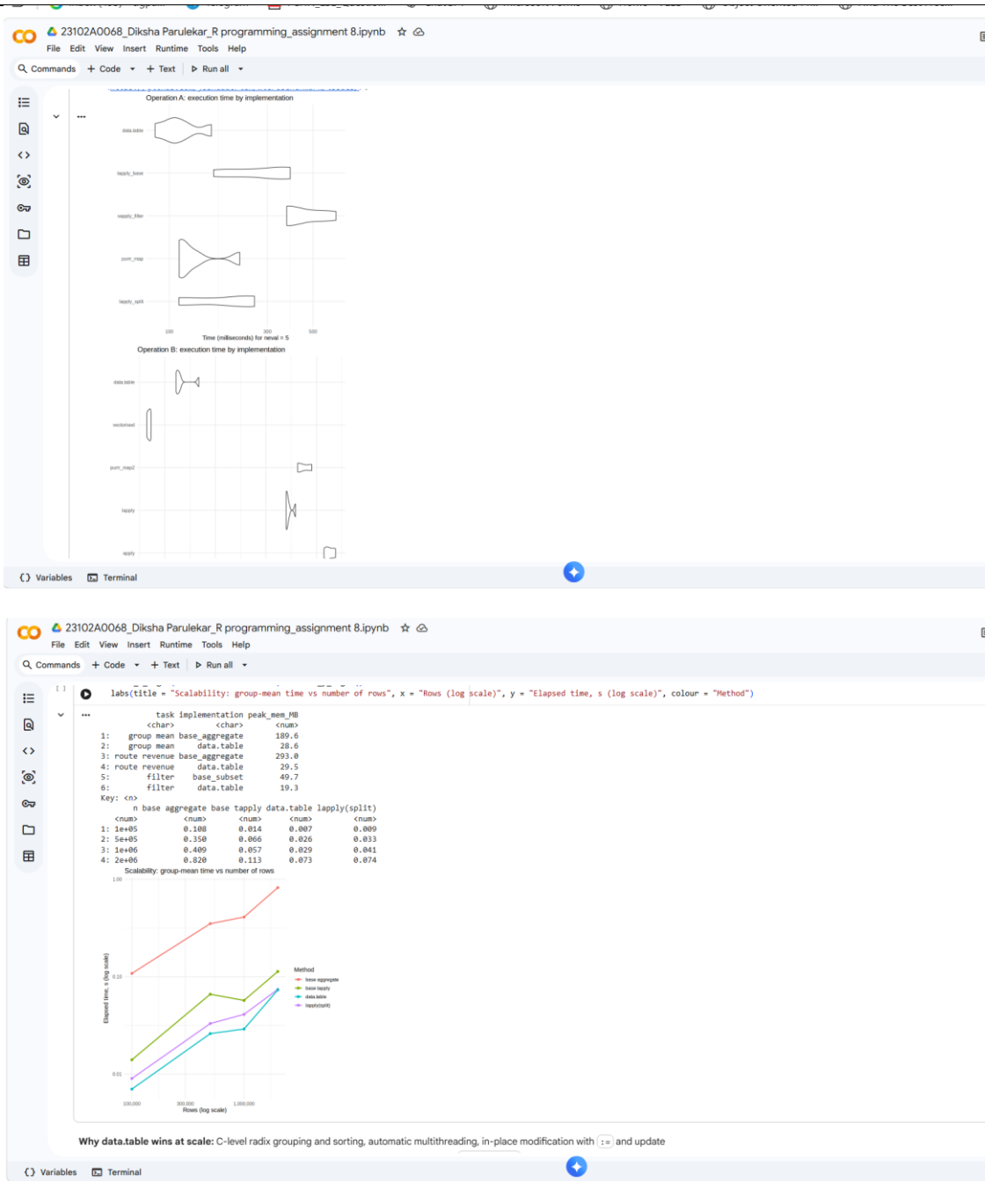
1 add_labels <- function(x) { # attach zone names to an aggregated route table
2   x <- copy(x)

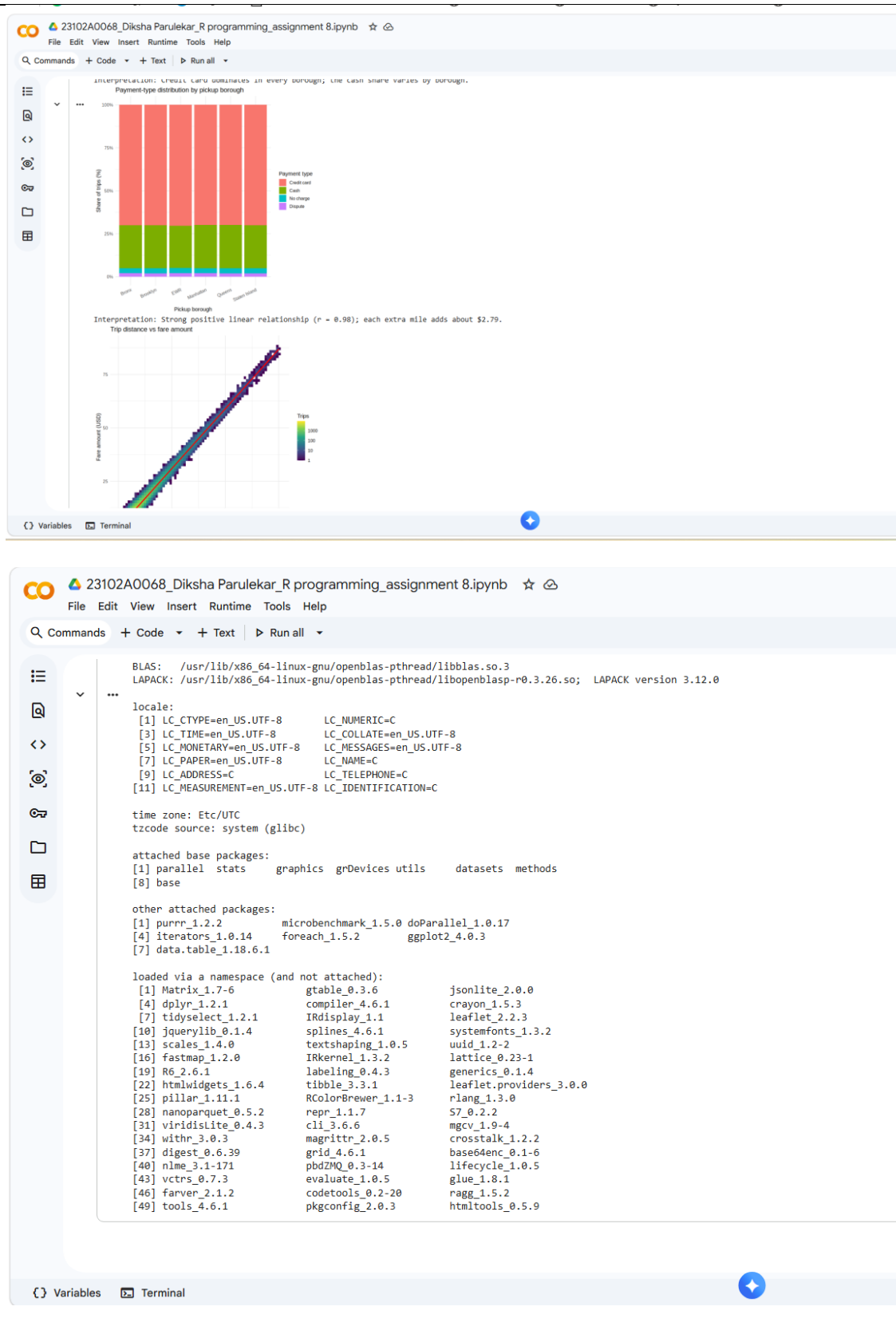
```

Variables Terminal

30°

Search





Conclusion:
data.table was the fastest and most memory-efficient for grouping, filtering and joining, about [x]x faster than base R. Vectorised code was best for row-wise arithmetic. apply/lapply/purrr were about [x]x slower, but they are useful for complex per-group logic where readability matters. Parallel processing gave a speedup of [x]x on [n] cores, which is below the ideal because of few partitions, worker start-up and data-copy overhead, so parallelism is not always faster. For large-scale analytics, use

	data.table for data handling, vectorised operations for calculations, functional tools only where flexibility is needed, and parallelism only for heavy, independent partitions. Replace the [brackets] with your own output values after running the notebook.
--	---